

Reflection Syntax Options Summary

Document #: P3419
Date: 2024-10-08
Project: Programming Language C++
Audience: EWG
Reply-to: Michael Levine
<mlevine55@bloomberg.net>

Contents

- 1 Goals
- 2 Some Potential Operator Pairs
- 3 Get Meta Operator Options
 - 3.1 Summary
 - 3.2 Advantages and Disadvantages
- 4 Splicing in Meta Operator Options
 - 4.1 Summary
 - 4.2 Advantages and Disadvantages
- 5 Get Meta Contextual and Function Keywords
 - 5.1 Summary
 - 5.2 Advantages and Disadvantages
- 6 Splice in Meta Contextual and Function Keywords
 - 6.1 Summary
 - 6.2 Advantages and Disadvantages
- 7 General Syntax Types
 - 7.1 Summary
 - 7.2 Advantages and Disadvantages
- 8 Acknowledgements
- 9 References

§ 1 Goals

This paper presents a summary of reflection syntax options. Some of these options are activately under consideration, while others have already been discussed and dismissed. However, the reasons for their dismissal have not necessary been recorded in a central location showcasing their perceived advantages or disadvantages relative to other options. This paper intends to be a reference for such discussions. A specific non-goal of the paper is the promotion of a particular syntax.

§ 2 Some Potential Operator Pairs

#	Get Meta	Splice in Meta
1	@<foo	@>foo
2	^^foo	[: foo :]
3	^^foo	, , foo
4	<-foo	->foo
5	\$\$foo	\$foo
6	`foo	, foo
7	^foo	[: foo :]
8	@<<foo	>>@foo
9	@<-foo	->@foo
10	<@foo	@>foo
11	<@foo	>@foo
12	&&foo	&&>foo
13	:foo	\$foo
14	/foo	\foo
15	^^foo	\$\$foo
16	``foo	, , foo
17	@<<foo	@>>foo
18	@<-foo	@->foo

§ 3 Get Meta Operator Options

§ 3.1 Summary

#	Syntax
1	^^foo
2	^foo
3	@<foo
4	@<<foo
5	@<-foo

#	Syntax
6	<@foo
7	foo
8	/foo
9	%foo
10	`foo
11	:foo
12	~foo
13	^_foo
14	<-foo
15	-<foo
16	^%foo
17	\$\$foo
18	&&foo
19	@foo
20	#foo
21	\$foo
22	'foo
23	=foo
24	?foo
25	\foo
26	^[foo]
27	\${foo}
28	\$(foo)
29	/\foo
30	``foo

§ 3.2 Advantages and Disadvantages

1: ^^foo

— Advantages

1. There is no ambiguity with blocks [P3381R0].
2. This is the syntax recommended by [P3381R0].

— Disadvantages

1. It is longer than the status quo [P3381R0].
2. It could potentially be used for logical xor.
3. Its use as a binary operator has a syntactic ambiguity with Clang's blocks extension and will probably not be able to be used for exclusive-or. An

example is `expr ^^ { return 1; }()`.

2: `^foo`

— Advantages

1. The upward arrow style notation indicates something is being pulled from `foo`.
2. It is a relatively rarely used binary operator [P3381R0].

— Disadvantages

1. Clang has an old extension called “blocks” that uses `^` to both introduce a block and as part of the syntax for “block types” [P3381R0]. With this extension, some parses may not be able to be disambiguated. `type-id(^ident)()` is ambiguous, and `auto A = ^{ f(); };` with [P3294R1] is ambiguous [P3381R0].
2. It is an operator being used in C++ for non-operator syntax.
3. It is too similar to `foo^`, which is used as a type in other languages built on top of C++ such as ObjectiveC++ [P3381R0] and the Microsoft .NET extensions.

3: `@<foo`

— Advantages

1. It is similar to Go’s channel syntax [GoChannels].
2. It shouldn’t have a nesting issue as the character order solves it.

— Disadvantages

1. There is potential concern about readability with the angle brackets in a template argument context, an example of which would be `fn<@<foo>`.

4: `@<<foo`

— Advantages

1. It makes the direction louder and easier to read than using `@<foo`.

5: `@<-foo`

— Advantages

1. It further emphasizes the direction of the data flow.

— Disadvantages

1. It might need parentheses for the `-` sign.

7: `|foo`

— Advantages

1. It is not part of a digraph [P3381R0].

2. This is not ambiguous to compilers [P3381R0].

— Disadvantages

1. It looks like half of an absolute value notation and could be confused as part of a magnitude notation when using multiple reflections in a single line as in `if (|T == |U)` [P3381R0].

8: /foo

— Advantages

1. It can offer nice symmetry with using `\` for code injection [P3381R0].

— Disadvantages

1. It is very close to comment syntax and could require parentheses for `/(/e)` and `/( *e)` [P3381R0].

9: %foo

— Advantages

1. It is a relatively rarely used binary operator [P3381R0].

— Disadvantages

1. When passed as a first template argument as in `C<%T> c;`, `<%` is a digraph for `{`. This would require spaces or parentheses to get around this issue [P3381R0].

10: `foo

— Advantages

1. This is a very lispy syntax. In Common Lisp it is used to disable evaluation for a subexpression [CLSpec].
2. It was recently added to the basic character set [P3381R0].
3. It is a small character, even smaller than `^` [P3381R0].

— Disadvantages

1. Markdown uses it extensively, and not all Markdown implementations give you ways to escape it [P3381R0].

11: :foo

— Advantages

1. It is used by the Julia programming language for metaprogramming to construct inline expressions such as `ex2 = Expr(:call, :+, 1, 1)` to produce a Julia Expr type such as `:(1 + 1)` [JuliaMeta].

— Disadvantages

1. Passing it as a template argument as in `C<:T>` would not work because `<:` is a digraph for `[` [P3381R0]. This can potentially be remedied by being called as `C<(:T)>`, but this could lead to errors that are difficult to debug.
2. It composes poorly with the splicing operator `[::foo:]`.
3. It may be ambiguous when parsing an array-index whose operand is a qualified-id or when parsing a reflect-expression composed inside a splice-specifier.

12: `~~foo`

— Disadvantages

1. It is ambiguous. If `foo` is an integral expression, then `~~foo` means to flip all of the bits and then flip them again.
2. It may require parentheses to parse the intended, single `~` symbol.
3. `~` is already a unary operator.

13: `^-foo`

— Disadvantages

1. It may require parentheses to parse the `-` symbol.

14: `<-foo`

— Advantages

1. It is similar to Go's channel syntax [GoChannels].

— Disadvantages

1. It could be hard to disambiguate `<<-` from `< <-` when used as a template argument as in `fn<<-foo>`.
2. It may require parentheses to parse the `-` symbol.

18: `&&foo`

— Disadvantages

1. It can lead to code like this: `if (&&foo&& == &&foo&& && &&foo&& == &&foo&&)` where `&&foo&&` is a reflection on a `foo` reflection type. This is comparing two reflections for equality.
2. It can be an issue with reflecting on an rvalue reference as in `(&&foo&&)`.
3. The `&&` is already used for logical AND and rvalue/forward references.

19: `@foo`

— Disadvantages

1. It was potentially going to be used as a splat for simple splices.

2. Compilers already allow the use of `@` in identifiers as an extension, so we wouldn't be able to disambiguate between reflecting on an identifier and actually being an identifier [P3381R0].
3. It is used as an extension in Objective-C. As such, a number of potential identifiers have meaning already (`@property`, `@dynamic`, `@optional`, etc). This also cannot be escaped since `@(foo)` is a boxed expression and `@[foo]` and `@{foo}` are container literals [P3381R0].

20: `#foo`

— Disadvantages

1. It precludes usage in existing C macros, given that `#define MACRO(x) do_something_with(#x)` already has meaning [P3381R0].

21: `$foo`

— Advantages

1. It doesn't conflict with anything else in C++ since it was recently added to the basic character set [P3381R0].

— Disadvantages

1. Compilers support its use in identifiers as an extension [P3381R0].
2. It is typically used as a splicing syntax in other languages, not as the get meta syntax.

23: `=foo`

— Advantages

1. It is already used as a binary operator [P3381R0].

— Disadvantages

1. It is too commonly used as a binary operator to now overload it for reflection usage [P3381R0].

24: `?foo`

— Advantages

1. It already exists within the language today [P3381R0].
2. It would not be ambiguous with the conditional operator [P3381R0].
3. The syntax naturally implies asking what an entity is, and the answer is its reflection [P3381R0].

— Disadvantages

1. It is associated with predicates in different programming languages for null coalescing, optional chaining, and error propagation (`v.empty?`) [P3381R0].

2. [P2688R1] proposes using it for pattern matching [P3381R0].

25: `\foo`

— Advantages

1. It is an available operator.

— Disadvantages

1. There is an issue with UCNs. `\u0` is parsed as a UCN instead of a reflection of `u0` [P3381R0].
2. Some prefer it as an interpolator, for which there is prior art [P3381R0].

26: `^[foo]`

— Advantages

1. It doesn't have an issue with block ambiguity, as opposed to `^foo` [P3381R0].
2. It can be more symmetric with splicing if splicing uses similar syntax like `[ :foo :]` [P3381R0].

— Disadvantages

1. It is a heavier syntax [P3381R0].

27: `${foo}`

— Advantages

1. It does not conflict with compiler support for using `$` in identifiers as an extension [P3381R0].

— Disadvantages

1. The `$` symbol is typically used as a splicing syntax in other languages, not as the get meta syntax.
2. There might be an issue with token sequences that might require an extra set of braces [P3381R0].

28: `$(foo)`

— Advantages

1. It does not conflict with compiler support for using `$` in identifiers as an extension [P3381R0].

— Disadvantages

1. The `$` is typically used as a splicing syntax in other languages, not as the get meta syntax.

29: `/\foo`

— Advantages

1. It is the big caret equivalent of the `^` symbol.

— Disadvantages

1. There is an issue with UCNs. `/\u0` will have `\u0` parsed as a UCN instead of a reflection of `u0` [P3381R0].

30: ``foo`

— Advantages

1. This is a very lispy syntax. In Common Lisp the single ``foo` is used to disable evaluation for a subexpression [CLSpec].

— Disadvantages

1. Markdown uses three backquotes to indicate the start and end of a code block, so it might be hard for Markdown to distinguish between starting a code block and capturing an inline code block of a reflection of `foo`.

§ 4 Splicing in Meta Operator Options

§ 4.1 Summary

#	Syntax
1	<code>[: foo :]</code>
2	<code>@>foo</code>
3	<code>@>>foo</code>
4	<code>@->foo</code>
5	<code>foo@></code>
6	<code>(: foo :)</code>
7	<code>(^ foo ^)</code>
8	<code>(^ _ foo _ ^)</code>
9	<code>->foo</code>
10	<code>>-foo</code>
11	<code>\$foo</code>
12	<code>\$\$foo</code>
13	<code>,foo</code>
14	<code>,,foo</code>
15	<code>&&>foo</code>
16	<code>foo@</code>
17	<code>[foo]</code>
18	<code>\${foo}</code>

#	Syntax
19	\$ (foo)
20	,@foo
21	foo&&
22	>>@foo
23	->@foo

§ 4.2 Advantages and Disadvantages

6: (: foo :)

— Disadvantages

1. It might be confusing to someone reading the code quickly.
2. It will render as a smiley face.

9: ->foo

— Advantages

1. It is similar to Go's channel syntax [[GoChannels](#)].

— Disadvantages

1. It could be a good relocation syntax in C++29.
2. It could be difficult to parse, particularly as a template argument as in `fn<->foo>`.

10: >-foo

— Disadvantages

1. There is a potential parsing ambiguity. `tvar<>-foo>-3` can be parsed as `(tvar<> - foo) > -3` or as `tvar<(>-foo)> - 3`.
2. It might require parentheses to parse the `-` symbol for code such as `>-(-foo)`.

11: \$foo

— Advantages

1. This syntax is used for splicing in a number of programming languages.
2. It doesn't conflict with anything else in C++ since it was recently added to the basic character set [[P3381R0](#)].

— Disadvantages

1. Compilers allow using `$` in identifiers as an extension, so it can be hard to differentiate between an identifier and reflecting on an identifier [[P3381R0](#)].

13: ,foo

— Advantages

1. In Common Lisp it is used to evaluate and splice in an expression [[CLSpec](#)].

— Disadvantages

1. It may be difficult to parse.

14: `,,foo`

— Disadvantages

1. It may be difficult to parse. An example could be `fn(,,bool, ,,int, ,,char)`.

17: `[| foo |]`

— Advantages

1. It is used in Template Haskell for expression quotation [[thQuotes](#)].

18 `${foo}`

— Advantages

1. It does not conflict with compiler support for using `$` in identifiers as an extension [[P3381R0](#)].

— Disadvantages

1. There might be an issue with token sequences that might require an extra set of braces [[P3381R0](#)].

19 `$(foo)`

— Advantages

1. It does not conflict with compiler support for using `$` in identifiers as an extension [[P3381R0](#)].

20: `,@foo`

— Advantages

1. In Common Lisp it is used to evaluate and splice in a list of values resulting from an expression [[CLSpec](#)].

§ 5 Get Meta Contextual and Function Keywords

§ 5.1 Summary

#	Syntax
1	lift foo
2	lift(foo)
3	reflexpr foo
4	reflexpr(foo)
5	refl foo
6	refl(foo)
7	ref foo
8	ref(foo)
9	meta foo
10	meta(foo)
11	symbol foo
12	symbol(foo)
13	syntax foo
14	syntax(foo)
15	expr foo
16	expr(foo)
17	reflect foo
18	reflect(foo)
19	metafy foo
20	metafy(foo)
21	metify foo
22	metify(foo)
23	inspect foo
24	inspect(foo)
25	reflectof foo
26	reflectof(foo)
27	reflof foo
28	reflof(foo)
29	metaof foo
30	metaof(foo)
31	std::meta::meta(foo)
32	std::meta::get(foo)
33	std::meta::parse(foo)

§ 5.2 Advantages and Disadvantages

4: reflexpr(foo)

— Advantages

1. This is the originally proposed keyword [P1240R0].

15: `expr foo`

— Disadvantages

1. Compilers often use internal `Expr` data structure definitions.
2. It implies getting an expression from `foo`, not meta info.
3. The full `expr foo` *would* be an expression, and `foo` is not generally an expression.

16: `expr(foo)`

— Disadvantages

1. Compilers often use internal `Expr` data structure definitions.
2. It implies getting an expression from `foo`, not meta info.

§ 6 Splice in Meta Contextual and Function Keywords

§ 6.1 Summary

#	Syntax
1	<code>splice foo</code>
2	<code>splice(foo)</code>
3	<code>splice_in foo</code>
4	<code>splice_in(foo)</code>
5	<code>eval foo</code>
6	<code>eval(foo)</code>
7	<code>evalmeta foo</code>
8	<code>evalmeta(foo)</code>
9	<code>unlift foo</code>
10	<code>unlift(foo)</code>
11	<code>demeta foo</code>
12	<code>demeta(foo)</code>
13	<code>unmeta foo</code>
14	<code>unmeta(foo)</code>
15	<code>unref foo</code>
16	<code>unref(foo)</code>
17	<code>deref foo</code>
18	<code>deref(foo)</code>
19	<code>unrefl foo</code>

#	Syntax
20	unrefl(foo)
21	derefl foo
22	derefl(foo)
23	value foo
24	value(foo)
25	getvalue foo
26	getvalue(foo)
27	unexpr foo
28	unexpr(foo)
29	deexpr foo
30	deexpr(foo)
31	dereflect foo
32	dereflect(foo)
33	unreflect foo
34	unreflect(foo)
35	demetafy foo
36	demetafy(foo)
37	unmetafy foo
38	unmetafy(foo)
39	unmetify foo
40	unmetify(foo)
41	uninspect foo
42	uninspect(foo)
43	deinspect foo
44	deinspect(foo)
45	dump foo
46	dump(foo)
47	unsyntax foo
48	unsyntax(foo)
49	desyntax foo
50	desyntax(foo)
51	desugar foo
52	desugar(foo)
53	unsymbol foo
54	unsymbol(foo)
55	desymbol foo
56	desymbol(foo)
57	unreflexpr foo

#	Syntax
58	<code>unreflexpr(foo)</code>

§ 6.2 Advantages and Disadvantages

17: `deref foo`

— Disadvantages

1. It sounds like you are dereferencing `foo`, not splicing in its meta value.

18: `deref(foo)`

— Disadvantages

1. It sounds like you are dereferencing `foo`, not splicing in its meta value.

58: `unreflexpr(foo)`

— Advantages

1. The original reflection design used this [P1240R0].

— Disadvantages

1. It is verbose [P3381R0].

§ 7 General Syntax Types

§ 7.1 Summary

#	Syntax Type
1	Unary prefix operators
2	Bounded splice operators
3	Single character splice operators
4	Function keywords
5	Contextual keywords
6	Library functions

§ 7.2 Advantages and Disadvantages

1: Unary prefix operators

— Disadvantages

1. It requires operator precedence rules to parse the code.
2. It may require inserting parentheses to ensure operational order, similar to pointer dereferencing: `*(foo())`.

2: Bounded splice operators

— Advantages

1. They can be easier to parse for nested name specifiers like `[:Cls::reflection_member:]::inner::member`, which is easier to parse than `$Cls::reflection_member::inner::member`.

3: Single character splice operators

— Advantages

1. They are a terser syntax.

— Disadvantages

1. There is potentially an issue with postfix operators when it comes to parsing.
2. It can be difficult to parse complicated expressions that need to be spliced in.

4: Function keywords

— Disadvantages

1. It is a heavier syntax, and it does not contribute more than a terser syntax to readability [P3381R0].
2. A reflection is typically taken to pass the reflection to a function that operates on it, not just to get the reflection. As a result, making a reflection syntax stand out is less impactful given its pass-through nature [P3381R0].
3. Novices will rarely use reflection, so choosing a novice-friendly keyword may not serve its most common user base [P3381R0].
4. For splicing, unparenthesized keywords, when used as a part of an unqualified-id, make it difficult to know where the splice ends. An example is `splice foo::bar::baz`.

5: Contextual keywords

— Disadvantages

1. It is a heavier syntax, and it does not contribute more than a terser syntax to readability [P3381R0].
2. A reflection is typically taken to pass the reflection to a function that operates on it, not just to get the reflection. As a result, making a reflection syntax stand out is less impactful given its pass-through nature [P3381R0].
3. Novices will rarely use reflection, so choosing a novice-friendly keyword may not serve its most common user base [P3381R0].

§ 8 Acknowledgements

Some of the syntax ideas, as well as their respective advantages and disadvantages, are my own, but many others came from discussions with Alisdair Meredith and a number of my other colleagues and peers.

§ 9 References

[CLSpec] Common Lisp HyperSpec: 2.4.6 Backquote.

https://www.lispworks.com/documentation/HyperSpec/Body/02_df.htm

[GoChannels] Go by Example: Channel Directions.

<https://gobyexample.com/channel-directions>

[JuliaMeta] Julia Programming Language Documentation: Metaprogramming.

<https://docs.julialang.org/en/v1/manual/metaprogramming/>

[P1240R0] Andrew Sutton, Faisal Vali, and Daveed Vandevoorde. 2018-08-10. Scalable Reflection in C++. (August 2018).

<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p1240r0.pdf>

[P2688R1] Michael Park. 2024-02-15. Pattern Matching: match Expression. (February 2024).

<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p2688r1.pdf>

[P3294R1] Andrei Alexandrescu, Barry Revzin, and Daveed Vandevoorde. 2024-07-16. Code Injection with Token Sequences. (July 2024).

<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p3294r1.html>

[P3381R0] Wyatt Childers, Peter Dimov, Dan Katz, Barry Revzin, Andrew Sutton, Faisal Vali, and Daveed Vandevoorde. 2024-09-16. Syntax for Reflection. (September 2024).

<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p3381r0.html>

[thQuotes] Glasgow Haskell Compiler Documentation: Language Extension: Template Haskell: Quotes Syntax.

https://downloads.haskell.org/ghc/latest/docs/users_guide/xts/template_haskell.html#extension-TemplateHaskellQuotes